

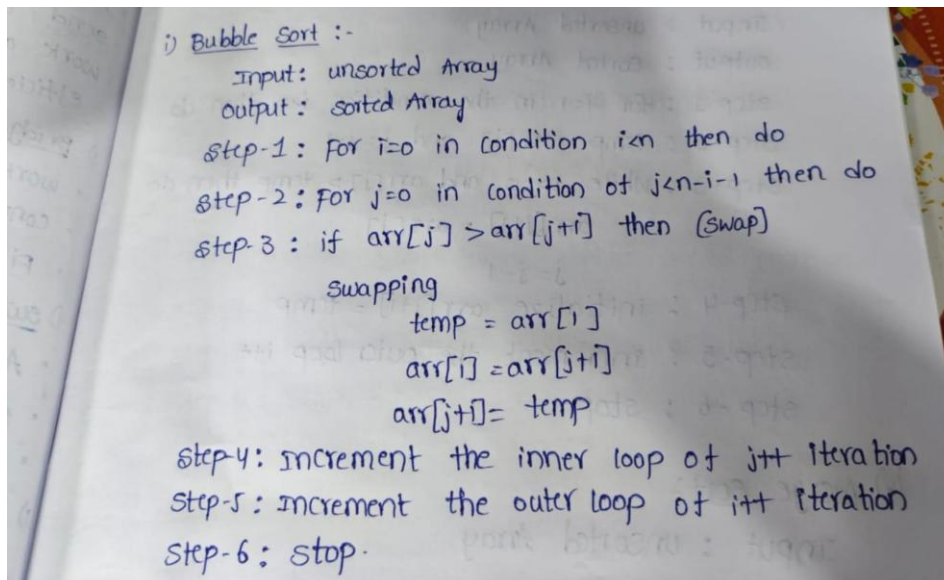
PRACTICAL-01

AIM:

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

BUBBLE SORT:

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;
class bubblesort
{
public:
    int size, arr[100], i;
    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;
        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
};
```

```
    }  
}  
void printarray()  
{  
    for (i = 0; i < size; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}  
void bubblelogic()  
{  
    int j, temp;  
    for (i = 0; i < size - 1; i++)  
    {  
        for (j = 0; j < size - i - 1; j++)  
        {  
            if (arr[j] > arr[j + 1])  
            {  
                temp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = temp;  
            }  
        }  
    }  
}  
};  
int main()  
{  
    bubblesort bs;  
    bs.arrayinput();  
    cout << "Before sorting: ";  
    bs.printarray();  
    bs.bubblelogic();  
    cout << "After sorting: ";  
    bs.printarray();  
    return 0;  
}
```

OUTPUT:

```
Output
Enter array size: 4
Enter array elements:
22
59
20
54
Before sorting: 22 59 20 54
After sorting: 20 22 54 59

=== Code Execution Successful ===
```

TIME COMPLEXITY:

Bubble Sort: Best case
first pass $\rightarrow n-1$
no swapping
loops stops
 $T(n) = n-1$
 $T(n) = O(n)$

average case:

Bubble Sort:

```
for (i=0; i<n-1; i++) {  
    for (j=0; j<n-i-1; j++) {  
        if (arr[j] > arr[j+1]) {  
            temp = arr[j];  
            arr[j] = arr[j+1];  
            arr[j+1] = temp;  
        }  
    }  
}
```

$T(n) = (n-1)(n-1)$
 $= n^2 - 2n + 1$
 $= n^2 - n$
 $T(n) = O(n^2)$

worst case:

if array is in reverse order it takes many comparisons and swaps

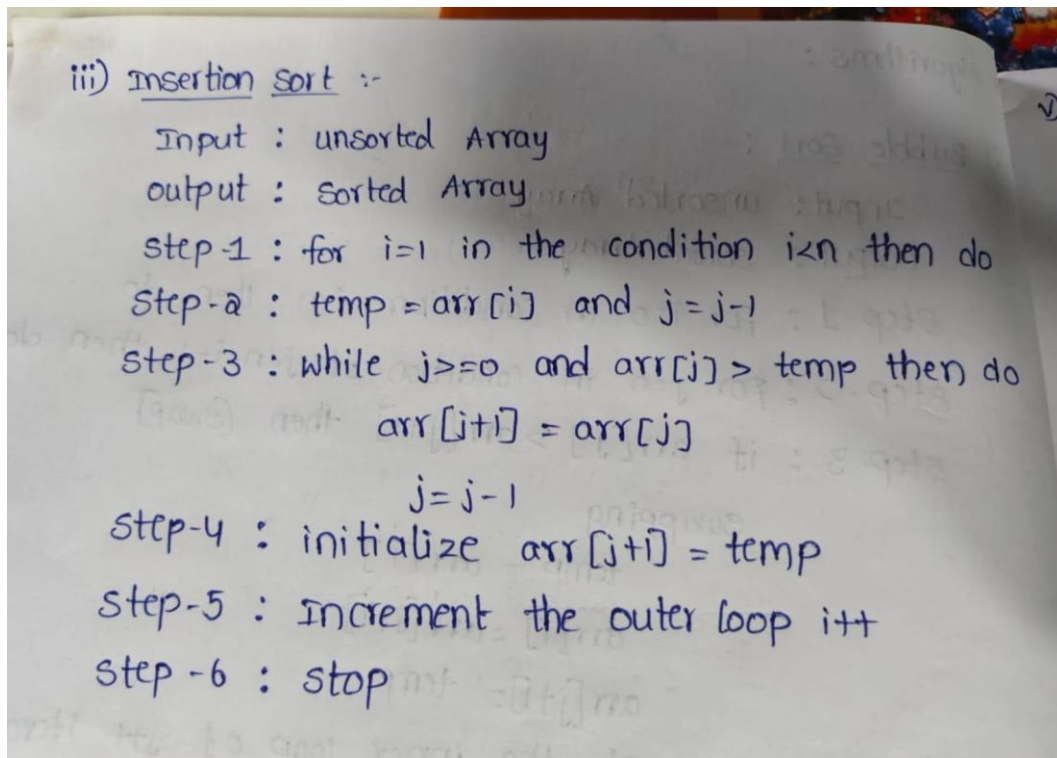
Bubble Sort:

```
for (i=0; i<n-1; i++) {  
    for (j=0; j<n-i-1; j++) {  
        if (arr[j] > arr[j+1]) {  
            temp = arr[j];  
            arr[j] = arr[j+1];  
            arr[j+1] = temp;  
        }  
    }  
}
```

$T(n) = (n-1)(n-1)$
 $= n^2 - 2n + 1$
 $= n^2 - n$
 $T(n) = O(n^2)$

INSERTION SORT:

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;

class InsertionSort
{
public:
    int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
```

```
        cin >> arr[i];
    }
}

void printarray()
{
    for (i = 0; i < size; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void insertionsortlogic()
{
    for (i = 1; i < size; i++)
    {
        temp = arr[i];
        j = i - 1;

        while (j >= 0 && arr[j] > temp)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = temp;
    }
}

};

int main()
{
    InsertionSort is;

    is.arrayinput();

    cout << "Before sorting: ";
    is.printarray();

    is.insertionsortlogic();

    cout << "After sorting: ";
    is.printarray();
    return 0;
}
```

}

OUTPUT:

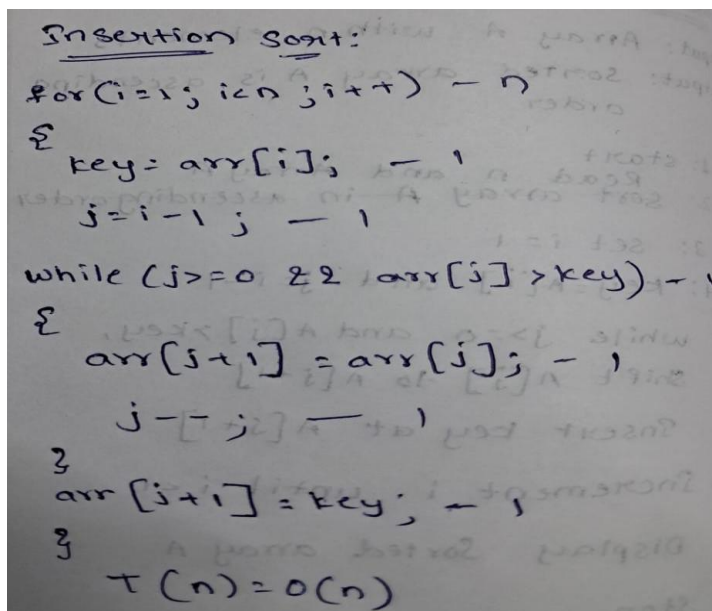
```
Output
Enter array size: 3
Enter array elements: 54
21
78
Before sorting: 54 21 78
After sorting: 21 54 78

=== Code Execution Successful ===
```

TIME COMPLEXITY:

best case:

if array is sorted we need only one comparison



average case:

Insertion Sort:
Average case:
pass - 1 $\rightarrow$ $n-1$ comparisons
pass - 2 $\rightarrow$ $n-2$ comparisons
$$T(n) = \frac{n(n-1)}{2}$$

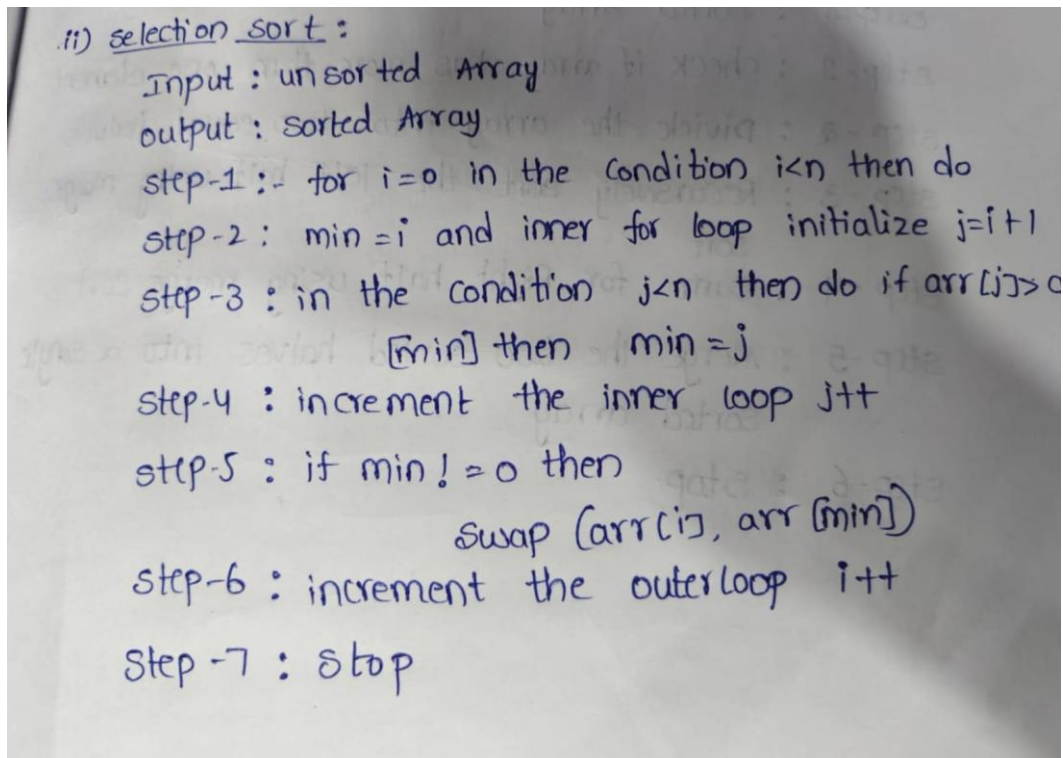
~~ignore~~ ignore constants
$$T(n) = O(n^2)$$

worst case:

the worst case occurs when
pivot always smallest (or) largest
$$n + (n-1) + (n-2) + \dots + 1$$
$$T(n) = \frac{n(n-1)}{2}$$
$$T(n) = O(n^2)$$

SELECTION SORT:

ALGORITHM:



ii) selection sort :

Input : unsorted Array
output : sorted Array

step-1 :- for $i=0$ in the condition $i < n$ then do

step-2 : $min = i$ and inner for loop initialize $j = i + 1$

step-3 : in the condition $j < n$ then do if $arr[j] < arr[min]$ then $min = j$

step-4 : increment the inner loop $j++$

step-5 : if $min \neq i$ then
swap ($arr[i], arr[min]$)

step-6 : increment the outer loop $i++$

step-7 : stop

CODE:

```
#include <iostream>
using namespace std;

class SelectionSort
{
public:
    int size, arr[100], i, j, temp;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;

        cout << "Enter array elements: ";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
}
```

```
    }

    void printarray()
    {
        for (i = 0; i < size; i++)
        {
            cout << arr[i] << " ";
        }
        cout << endl;
    }

    void selectionsortlogic()
    {
        for (i = 0; i < size - 1; i++)
        {
            int min = i;

            for (j = i + 1; j < size; j++)
            {
                if (arr[j] < arr[min])
                {
                    min = j;
                }
            }

            if (min != i)
            {
                temp = arr[i];
                arr[i] = arr[min];
                arr[min] = temp;
            }
        }
    }
};

int main()
{
    SelectionSort ss;

    ss.arrayinput();

    cout << "Before sorting: ";
    ss.printarray();

    ss.selectionsortlogic();

    cout << "After sorting: ";
    ss.printarray();
}
```

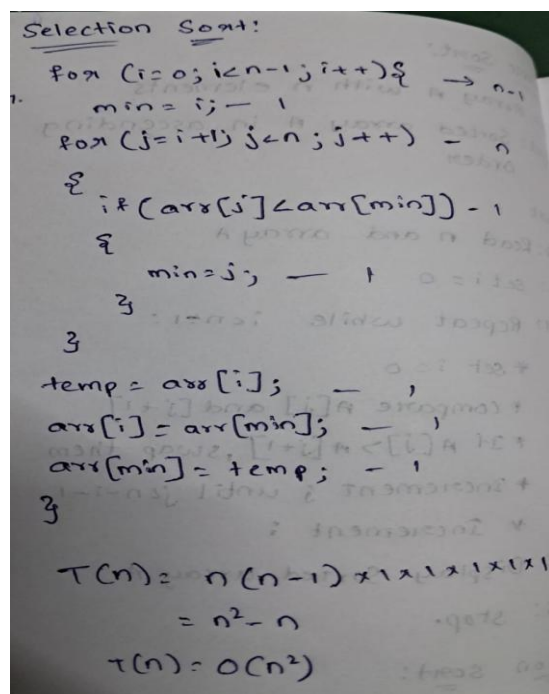
```
    return 0;  
}
```

OUTPUT:

```
Output  
Enter array size: 3  
Enter array elements: 21  
65  
  
3  
Before sorting: 21 65 3  
After sorting: 3 21 65  
  
=== Code Execution Successful ===
```

TIME COMPLEXITY:

best case:





average case:

Selection Sort:

```
for (i=0; i<n-1; i++) {  
    min = i; - 1  
    for (j=i+1; j<n; j++) - n  
    {  
        if (arr[j] < arr[min]) - 1  
        {  
            min = j; - 1  
        }  
    }  
    temp = arr[i];  
    arr[i] = arr[min];  
    arr[min] = temp;  
}
```

$$T(n) = n(n-1) \times 1 \times 1 \times 1 \times 1$$
$$= n^2 - n$$
$$T(n) = O(n^2)$$

worst case:

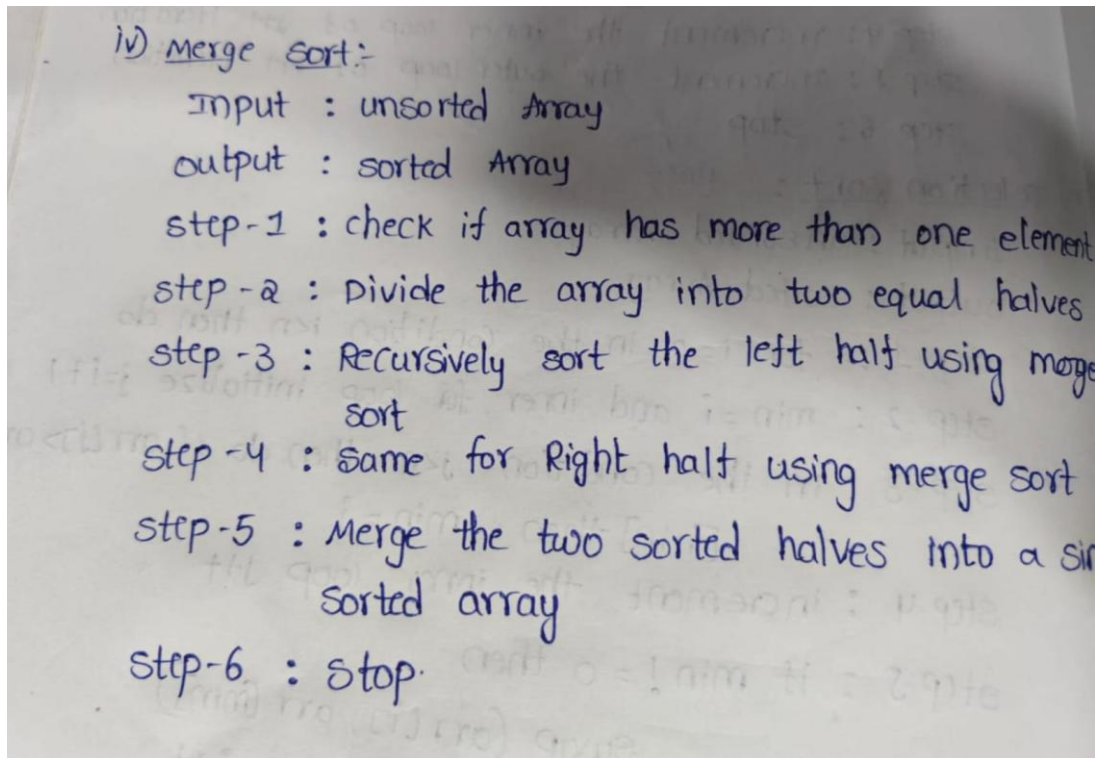
Selection Sort:

```
for (i=0; i<n-1; i++) {  
    min = i; - 1  
    for (j=i+1; j<n; j++) - n  
    {  
        if (arr[j] < arr[min]) - 1  
        {  
            min = j; - 1  
        }  
    }  
    temp = arr[i];  
    arr[i] = arr[min];  
    arr[min] = temp;  
}
```

$$T(n) = n(n-1) \times 1 \times 1 \times 1 \times 1$$
$$= n^2 - n$$
$$T(n) = O(n^2)$$

MERGE SORT:

ALGORITHM:



CODE:

```
#include <iostream>
using namespace std;

class mergesort
{
public:
    int size, arr[100], i;

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;
        cout << "Enter array elements:\n";
        for (i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
};
```

```
    }  
}  
void printarray()  
{  
    for (i = 0; i < size; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}  
  
void merge(int low, int mid, int high)  
{  
    int temp[100];  
    int i = low, j = mid + 1, k = low;  
  
    while (i <= mid && j <= high)  
    {  
        if (arr[i] <= arr[j])  
        {  
            temp[k] = arr[i];  
            i++;  
        }  
        else  
        {  
            temp[k] = arr[j];  
            j++;  
        }  
        k++;  
    }  
  
    while (i <= mid)  
    {  
        temp[k] = arr[i];  
        i++;  
        k++;  
    }  
  
    while (j <= high)  
    {  
        temp[k] = arr[j];  
        j++;  
        k++;  
    }  
}
```

```
        for (i = low; i <= high; i++)
        {
            arr[i] = temp[i];
        }
    }

    void mergellogic(int low, int high)
    {
        if (low < high)
        {
            int mid = (low + high) / 2;

            mergellogic(low, mid);
            mergellogic(mid + 1, high);

            merge(low, mid, high);
        }
    }
};

int main()
{
    mergesort ms;

    ms.arrayinput();

    cout << "Before sorting: ";
    ms.printarray();

    ms.mergellogic(0, ms.size - 1);

    cout << "After sorting: ";
    ms.printarray();

    return 0;
}
```

OUTPUT:

```
Enter array size: 3
Enter array elements:
21
5
69
Before sorting: 21 5 69
After sorting: 5 21 69

=== Code Execution Successful ===
```

TIME COMPLEXITY:

best case, average case and worst case

Merge Sort

```

void merge (int low, int mid, int high)
{
    int temp[100];
    int i = low, j = mid + 1, k = low;
    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
        }
        else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }
    while (j <= high) {
        temp[k] = arr[j];
        j++;
        k++;
    }
    for (i = low; i <= high; i++)
        arr[i] = temp[i];
}

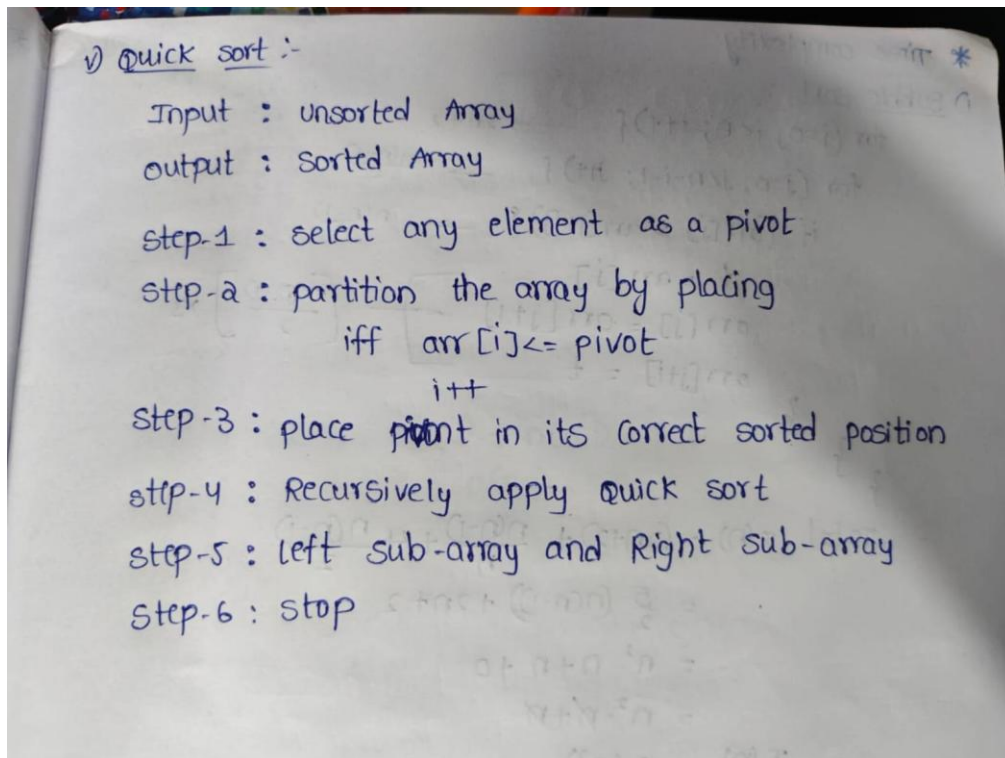
void mergeLogic (int low, int high)
{
    if (low < high)
        mid = (low + high) / 2;
    mergeLogic (low, mid);
    mergeLogic (mid + 1, high);
    merge (low, mid, high);
}
    
```



$$\begin{aligned}
 T(n) &= T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n \\
 T(n) &= 2T\left(\frac{n}{2}\right) + n \quad \text{--- (1)} \\
 T\left(\frac{n}{2}\right) &= 2T\left(\frac{n}{4}\right) + \frac{n}{2} \quad \text{--- (2)} \\
 \text{Sub (2) in (1)} \\
 T(n) &= 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n \\
 T(n) &= 4T\left(\frac{n}{4}\right) + 2n \quad \text{--- (3)} \\
 T\left(\frac{n}{4}\right) &= 2T\left(\frac{n}{8}\right) + \frac{n}{4} \quad \text{--- (4)} \\
 \text{Sub (4) in (3)} \\
 T(n) &= 4\left(2T\left(\frac{n}{8}\right) + \frac{n}{4}\right) + 2n \\
 &= 8T\left(\frac{n}{8}\right) + 2n + 2n \\
 &= 8T\left(\frac{n}{8}\right) + 4n \\
 T(n) &= 8T\left(\frac{n}{8}\right) + 4n \\
 &= 2^4 T\left(\frac{n}{2^4}\right) + 4n \\
 T(n) &= 2^k T\left(\frac{n}{2^k}\right) + kn \\
 \frac{n}{2^k} &= 1 \\
 n &= 2^k \\
 \text{Apply log on both sides} \\
 \log n &= \log 2^k \\
 \log n &= k \log 2 \\
 \log n &= k \\
 T(n) &= 2^{\log n} T\left(\frac{n}{2^{\log n}}\right) + n \\
 T(n) &= 2^{\log n} (1) + n \log n \\
 T(n) &= n \log^2 + n \log n \\
 T(n) &= n(1) + n \log n \\
 \boxed{T(n) &= n \log n}
 \end{aligned}$$

QUICK SORT:

ALGORITHM:



CODE:

```
#include <iostream>
#include <utility>
using namespace std;

class QuickSort
{
public:
    int size, arr[100];

    void arrayinput()
    {
        cout << "Enter array size: ";
        cin >> size;
        cout << "Enter array elements: ";
        for (int i = 0; i < size; i++)
        {
            cin >> arr[i];
        }
    }
};
```

```
    }  
}  
  
void printarray()  
{  
    for (int i = 0; i < size; i++)  
    {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
}  
  
int partition(int lb, int ub)  
{  
    int pivot = arr[lb];  
    int start = lb;  
    int end = ub;  
  
    while (start < end)  
    {  
        while (start <= ub && arr[start] <= pivot)  
            start++;  
  
        while (arr[end] > pivot)  
            end--;  
  
        if (start < end)  
            swap(arr[start], arr[end]);  
    }  
  
    swap(arr[lb], arr[end]);  
    return end;  
}  
  
void quicksortlogic(int lb, int ub)  
{  
    if (lb < ub)  
    {  
        int loc = partition(lb, ub);  
        quicksortlogic(lb, loc - 1);  
        quicksortlogic(loc + 1, ub);  
    }  
}  
  
void sort()
```

```
{
    quicksortlogic(0, size - 1);
}
};

int main()
{
    QuickSort qs;

    qs.arrayinput();

    cout << "Before sorting: ";
    qs.printarray();

    qs.sort();

    cout << "After sorting: ";
    qs.printarray();

    return 0;
}
```

OUTPUT:

Output

```
Enter array size: 3
Enter array elements: 25
65
8
Before sorting: 25 65 8
After sorting: 8 25 65
```

```
=== Code Execution Successful ===
```



TIME COMPLEXITY:

Best case:

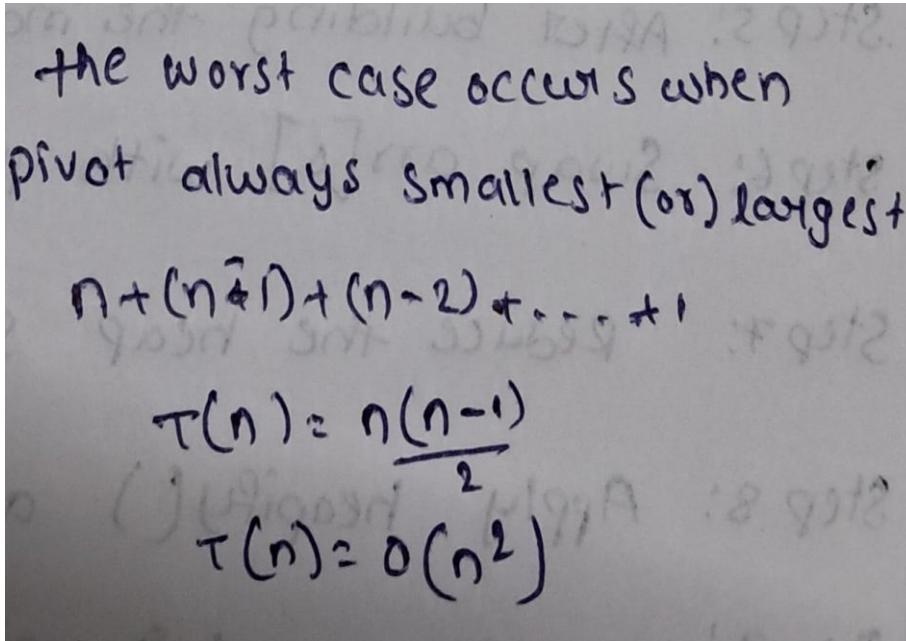
```
Quick Sort
if (low < high) - 1
{
    pivot = arr[high]; - 1
    i = low - 1; - 1
    for (j = low; j < high; j++) - n
    {
        if (arr[j] < pivot) - 1
        {
            i++; - 1
            temp = arr[i]; - 1
            arr[i] = arr[j]; - 1
            arr[j] = temp; - 1
        }
    }
    temp = arr[i+1]; - 1
    arr[i+1] = arr[high] - 1
    arr[high] = temp; - 1
    p = i+1;
    quick sort(arr, low, p-1); - 1
    quick sort(arr, p+1, high); - 1
}
```

$$\begin{aligned}
 T(n) &= 2T(n/2) + O(n) \quad \text{--- (1)} \\
 T(n/2) &= 2T(n/4) + \frac{n}{2} \quad \text{--- (2)} \\
 \text{sub (2) in (1)} \\
 T(n) &= 2\left(2T(n/4) + \frac{n}{2}\right) + n \\
 T(n) &= 4T(n/4) + n + n \\
 T(n) &= 4T(n/4) + 2n \\
 T(n) &= 2^k T\left(\frac{n}{2^k}\right) + kn \quad \begin{matrix} \frac{n}{2^k} = 1 \\ n = 2^k \end{matrix} \\
 \log n &= \log 2^k \\
 T(n) &= 2 \log^n(1) + \log n(n) \quad \log n = k \\
 T(n) &= n \log^2(1) + n \log n \\
 T(n) &= n + n \log n \\
 T(n) &= O(n \log n)
 \end{aligned}$$

average case:

$$\begin{aligned}
 T(n) &= 2T(n/2) + O(n) \quad \text{--- (1)} \\
 T(n/2) &= 2T(n/4) + \frac{n}{2} \quad \text{--- (2)} \\
 \text{sub (2) in (1)} \\
 T(n) &= 2\left(2T(n/4) + \frac{n}{2}\right) + n \\
 T(n) &= 4T(n/4) + n + n \\
 T(n) &= 4T(n/4) + 2n \\
 T(n) &= 2^k T\left(\frac{n}{2^k}\right) + kn \quad \begin{matrix} \frac{n}{2^k} = 1 \\ n = 2^k \end{matrix} \\
 \log n &= \log 2^k \\
 T(n) &= 2 \log^n(1) + \log n(n) \quad \log n = k \\
 T(n) &= n \log^2(1) + n \log n \\
 T(n) &= n + n \log n \\
 T(n) &= O(n \log n)
 \end{aligned}$$

worst case:



the worst case occurs when
pivot always smallest (or) largest
 $n + (n-1) + (n-2) + \dots + 1$
 $T(n) = \frac{n(n-1)}{2}$
 $T(n) = O(n^2)$

CONCLUSION:

Hence, By Implementing and analysing of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

The time complexity plays an important role in the logical part of the program. And the Programs are successfully executed.